

Hi,

Welcome to the next part of the data ingestion. In the previous module you have written some code to ingest data at-rest. This time you're going to deal with data in-motion, so with streaming pipelines!

Part 1 - API Gateway

Exercise 1

Your website visitors can opt-in or opt-out sharing their visit activity with you. Any agreement will generate data with the following fields:

```
Visit

{
  "eventId": string,
  "visitId": integer,
  "userId": integer,
  "visitedPage": string,
  "visitTime": timestamp
}
```

Your goal is to implement the data producer that will be running as a part of the API Gateway. The producer should:

- write all incoming events to *raw_data* topic in Apache Kafka
 - the writing process should use the batching strategy of Apache Kafka; there are 2 ([1](#), [2](#)) great resources explaining this concept
 - the producer should keep all events related to the visit in the same Apache Kafka partition; a visit is identified with the *visitId* attribute
- if the broker is unavailable, all undelivered events should be written to a local file system

It's more a data writing exercise. That's why we don't focus here on the API parts, such as responses or authentication.

Setup instructions

Java/Scala

Part 1 - Docker

1. Go to the Docker directory: `cd docker/p1/e1`
2. Start the Apache Kafka cluster: `docker-compose down; docker-compose up`

Part 2 - API Gateway

1. Open <https://github.com/bartosz25/master-stream-processing-homework/tree/master/01-data-ingestion-real-time/scala> in IntelliJ.
2. Implement the delivery logic in `com.becomedataengineer.exercise1.part1.api.KafkaDataProducer#produceDataToKafka` and `com.becomedataengineer.exercise1.part1.api.KafkaDataProducer#saveUndeliveredEvents`. If you want, you can reorganize the project classes!

Part 3 - Data generation

1. Go to the `com.becomedataengineer.exercise1.part1.api.ApiEntrypoint` object and run it.
2. Go to `com.becomedataengineer.exercise1.part1.DataGenerator` and run it.
3. The `DataGenerator` sends HTTP requests to the API from point (2) every 5 seconds.

Part 4 - Verify produced messages

1. Run `docker exec -ti e1_kafka_1 kafka-console-consumer.sh --bootstrap-server localhost:29092 --topic raw_data --from-beginning`. You should see the messages produced in the previous point.

Python

Part 1 - Docker

1. Go to the Docker directory: `cd docker/p1/e1`
2. Start the Apache Kafka cluster: `docker-compose down; docker-compose up`

Part 2 - API Gateway

1. Open <https://github.com/bartosz25/master-stream-processing-homework/tree/master/01-data-ingestion-real-time/python> in PyCharm.
2. Implement the delivery logic in `exercise1.part1.api.kafka_data_producer.KafkaDataProducer#produce_data_to_kafka` and `exercise1.part1.api.kafka_data_producer.KafkaDataProducer#save_undelivered_records`. If you want, you can reorganize the project classes!

Part 3 - Data generation

1. Go to `exercise1/part1/api/api_entrypoint.py` and run it.
2. Go to `exercise1/part1/generation/data_generator.py` and run it.
3. The `data_generator` sends HTTP requests to the API from point (2) every 5 seconds.

Part 4 - Verify produced messages

1. Run `docker exec -ti e1_kafka_1 kafka-console-consumer.sh --bootstrap-server localhost:29092 --topic raw_data --from-beginning`. You should see the messages produced in the previous point.

Part 2 - Batch layer

Exercise 1

In the Part 1 you set up a real-time data ingestion. As you know, real-time is not the single requirement in data systems. A lot of them work pretty fine with batch pipelines that rely on the data at-rest storage.

Your task is to extend the API Gateway project with an Apache Spark Structured Streaming writing the generated data as Delta Lake files to your local file system. The job should meet the following requirements:

- continuously process events coming to the `visits` topic
- generate Delta Lake files composed of the following fields:
 - `value` - a string field that should be read directly from the Apache Kafka topic
- classify data as *valid* and *invalid*
 - a record is considered *invalid* when the `event_time` column is missing
- valid records should be written to `/tmp/bde-3/homework/p2/e1/output/valid`
- invalid records should be written to `/tmp/bde-3/homework/p2/e1/output/invalid`
- partition the data so that it gets written into subdirectories of the following naming pattern: `year={year}/month={month}/day={day}/hour={hour}`
 - for the valid records, the partition values should be extracted from the `event_time` attribute
 - for the invalid records, the partition values should be extracted from the current time

The schema of the events written to the `visits` topic is:

field	type
eventId	string
visitId	integer
userId	integer
visitedPage	string
eventTime	timestamp

Setup instructions

Java/Scala

1. Open <https://github.com/bartosz25/master-stream-processing-homework/tree/master/01-data-ingestion-real-time/scala> in IntelliJ.
2. Implement your job.
3. Go to `com.becomedataengineer.exercise2.part1.DataGenerator` and run it.
4. The `DataGenerator` should continuously write 7 records to the visits topic.
5. You can check the content of the synchronized records by running the `com.becomedataengineer.exercise2.part1.SynchronizedRecordsReader`.

Python

1. Open <https://github.com/bartosz25/master-stream-processing-homework/tree/master/01-data-ingestion-real-time/python> in PyCharm.
2. Implement your job.
3. Go to `exercise2/part1/data_generator.py` and run it.
4. The `data_generator` should continuously write 7 records to the visits topic.
5. You can check the content of the synchronized records by running the `exercise2/part1/synchronized_records_reader.py`.

💡 **If you are stuck**, please check the *Homework - instructions* document present in the same page. It contains some extra details on how to implement various parts of the homework.

Have fun and see you in the next module!

Bartosz.